

ASWANNA: Complete System Engineering Blueprint

SRS Alignment, Dual Core Engines, Monorepo Layout, Docker Rules & Starter Implementation

Core Mission: ASWANNA targets the **30% to 40% post-harvest food loss** in Sri Lanka's central highlands (Uva Province / Bandarawela). It is an agricultural planning and loss mitigation platform operating via two core engines:

- **Phase 1 (Prevention Engine):** Predictive crop planning where farmers log intentions before sowing. The backend evaluates regional land quotas to avoid market over-saturation and price crashes.
- **Phase 2 (Recovery Engine):** Geo-fenced zero-waste surplus matching strictly within a **5km radius** to connect unexpected yields directly with local food processors, buyers, and livestock owners.

1. Data Architecture: Why Hybrid PostgreSQL & MongoDB Replaces Firebase

Architectural Decision: Firebase was replaced because NoSQL document stores lack transactional consistency for multi-farmer quota limits and cannot perform server-side trigonometric spatial calculations (Haversine math).

- **PostgreSQL 15:** Enforces relational ACID integrity for regional crop quotas, user authentication, and 5km geospatial SQL queries.
- **MongoDB 6.0:** Serves as a high-throughput caching and telemetry layer with a 30-minute TTL index for OpenWeather advisories and climate logs.
- **Socket.io / WebSockets:** Delivers real-time saturation alerts and surplus notifications to Flutter and React clients without Firebase lock-in.

2. Git Workflow & Docker Execution Policy

Team Docker Execution Rules:

- **Nirman (DevOps):** Builds and maintains docker-compose.yml and Kubernetes manifests.
- **Tharaka (Backend):** Runs containerized backend with live hot-reload volume binds (./backend:/app).
- **Ravindi (Frontend): No Docker required.** Runs Vite locally on the host OS (npm run dev) connecting to http://localhost:5000.
- **Imal (Mobile): No Docker required.** Runs Flutter natively on the host OS, pointing Android Emulator to http://10.0.2.2:5000/api/v1.
- **Miyuni (QA):** Runs automated Jest test suites and Postman collections locally against the exposed Docker backend port.

Git Branching Standards:

- **main (Protected):** Production-ready code. Direct commits are restricted.
- **develop (Protected):** Integration branch for completed and tested features.
- **Feature Branches:** Structured naming convention (e.g. feature/backend-quota-engine, feature/flutter-hive-storage).
- **CI Gate:** GitHub Actions blocks PR merges if linting or Jest unit tests fail.

3. Standardized Monorepo File & Directory Layout

```

aswanna-core/
├── .github/
│   └── workflows/
│       └── ci-pipeline.yml # Nirman: Automated PR test & lint workflow
├── backend/ # Tharaka: Layered Node.js API
│   ├── src/
│   │   ├── config/
│   │   │   ├── db.pg.js # PostgreSQL connection pool
│   │   │   └── db.mongo.js # MongoDB telemetry & TTL connection
│   │   ├── controllers/
│   │   │   ├── auth.controller.js # JWT generation & verification
│   │   │   ├── planning.controller.js # Pre-planting quota check & inserts
│   │   │   ├── surplus.controller.js # 5km Haversine geo-spatial matching
│   │   │   └── officer.controller.js # Divisional saturation queries
│   │   ├── middleware/
│   │   │   ├── auth.middleware.js # JWT Bearer validation
│   │   │   └── rbac.middleware.js # farmer / officer / buyer permissions
│   │   └── routes/
│   │       ├── api.routes.js # Route aggregator (/api/v1)
│   │       └── officer.routes.js
│   └── services/
│       ├── haversine.service.js # Geo-distance calculations
│       ├── weather.service.js # OpenWeather MongoDB TTL proxy
│       └── app.js # Express app instance export
├── tests/ # Miyuni: Automated test suites
│   ├── planning_and_surplus.test.js
│   └── rbac.test.js
├── Dockerfile
├── package.json
├── server.js # HTTP listener endpoint
├── frontend/ # Ravindi: React.js Vite Dashboard
│   ├── src/
│   │   ├── assets/
│   │   ├── components/
│   │   │   ├── QuotaProgressBar.jsx # Dynamic saturation visualizer
│   │   │   ├── SurplusCard.jsx # 5km buyer distress card
│   │   │   └── RoleNavbar.jsx
│   │   ├── pages/
│   │   │   ├── OfficerQuotaDashboard.jsx # Saturation heatmaps per crop
│   │   │   ├── FarmerPortal.jsx # Division intention status
│   │   │   └── BuyerSurplusFeed.jsx # 5km geo-discovery interface
│   │   ├── services/
│   │   │   └── api.js # Axios instance with Bearer interceptors
│   │   ├── App.jsx
│   │   └── main.jsx
│   ├── package.json
│   ├── tailwind.config.js
│   └── vite.config.js
├── mobile/ # Imal: Flutter Mobile App
│   ├── lib/
│   │   ├── bloc/
│   │   │   ├── planning_bloc.dart # Crop intent state management
│   │   │   └── surplus_bloc.dart # Distress surplus publish state
│   │   ├── models/
│   │   │   ├── crop_plan.dart
│   │   │   └── surplus_item.dart
│   │   ├── services/
│   │   │   ├── planning_service.dart # Dio API client
│   │   │   └── hive_storage.dart # Offline-first SQLite/Hive persistence
│   │   ├── views/
│   │   │   ├── pre_planting_view.dart # Intent form with saturation alert
│   │   │   ├── weather_advisory.dart # 30-min cached weather feed
│   │   │   └── emergency_surplus.dart # Instant 5km surplus dispatch
│   │   └── main.dart
│   └── pubspec.yaml
├── k8s/ # Nirman: Week 4 Kubernetes manifests
│   ├── backend-deployment.yaml
│   ├── postgres-statefulset.yaml
│   ├── .env.example # Shared environment template
│   ├── .gitignore
│   ├── docker-compose.yml # Master local multi-container setup
│   └── README.md # Developer onboarding guide

```

4. Member Implementation Plans & Starter Code

Nirman - DevOps & Infrastructure Lead

- **Step 1:** Configure Git repo with protected main and develop branches.
- **Step 2:** Build docker-compose.yml for PostgreSQL 15, MongoDB 6.0, and Node.js backend.
- **Step 3:** Configure .env.example with spatial limits (MAX_SURPLUS_RADIUS_KM=5).
- **Step 4:** Set up container volume mounts (./backend:/app) enabling hot-reloading.
- **Step 5:** Author GitHub Actions workflow executing linting and Jest test runs on Pull Requests.
- **Step 6:** Construct Kubernetes Deployment manifests for Week 4 production release.

```
version: '3.8'

services:
  postgres-db:
    image: postgres:15-alpine
    container_name: aswanna_postgres
    restart: unless-stopped
    environment:
      POSTGRES_USER: aswanna_admin
      POSTGRES_PASSWORD: SecretPassword123!
      POSTGRES_DB: aswanna_core
    ports:
      - "5432:5432"
    volumes:
      - pg_data:/var/lib/postgresql/data

  mongo-db:
    image: mongo:6.0
    container_name: aswanna_mongo
    restart: unless-stopped
    ports:
      - "27017:27017"
    volumes:
      - mongo_data:/data/db

  backend:
    build: ./backend
    container_name: aswanna_backend
    ports:
      - "5000:5000"
    environment:
      PORT: 5000
      POSTGRES_URL: postgres://aswanna_admin:SecretPassword123!@postgres-db:5432/aswanna_core
      MONGO_URL: mongodb://mongo-db:27017/aswanna_telemetry
      JWT_SECRET: aswanna_jwt_secret_2026
      MAX_SURPLUS_RADIUS_KM: 5
      WEATHER_API_KEY: ${WEATHER_API_KEY}
    depends_on:
      - postgres-db
      - mongo-db
    volumes:
      - ./backend:/app
      - /app/node_modules

volumes:
  pg_data:
  mongo_data:
```

Tharaka - Backend Lead (Node.js & Express)

- **Step 1:** Initialize Express app with PostgreSQL connection pooling and MongoDB client.
- **Step 2:** Build JWT authentication and RBAC middleware (farmer, officer, buyer).
- **Step 3:** Construct Crop Planning API (POST /api/v1/plans) with regional saturation logic.
- **Step 4:** Construct Divisional Quotas API (GET /api/v1/officer/quotas).
- **Step 5:** Implement 5km Geo-Fenced Surplus Matching using SQL Haversine spherical distance.
- **Step 6:** Build OpenWeather proxy service storing raw forecasts in MongoDB with a 30-min TTL.

```

const { Pool } = require('pg');
const pool = new Pool({ connectionString: process.env.POSTGRES_URL });

// 1. Submit Pre-Planting Crop Intent (Prevents Market Over-Saturation)
exports.createCropPlan = async (req, res) => {
  const { farmer_id, division_id, crop_type, planned_area_acres, expected_harvest_date } = req.body;

  const quotaCheck = await pool.query(
    'SELECT SUM(planned_area_acres) as current_total, quota_limit_acres ' +
    'FROM crop_plans WHERE division_id = $1 AND crop_type = $2 ' +
    'GROUP BY quota_limit_acres',
    [division_id, crop_type]
  );

  const currentTotal = quotaCheck.rows[0]?.current_total || 0;
  const quotaLimit = quotaCheck.rows[0]?.quota_limit_acres || 100;
  const isOverSaturationRisk = (Number(currentTotal) + Number(planned_area_acres)) > quotaLimit;

  const result = await pool.query(
    'INSERT INTO crop_plans ' +
    '(farmer_id, division_id, crop_type, planned_area_acres, expected_harvest_date, risk_flag) ' +
    'VALUES ($1, $2, $3, $4, $5, $6) RETURNING *',
    [farmer_id, division_id, crop_type, planned_area_acres, expected_harvest_date, isOverSaturationRisk]
  );

  res.status(201).json({
    plan: result.rows[0],
    warning: isOverSaturationRisk ? 'High risk of overproduction in this division for this harvest window!' : null
  });
};

// 2. Query Surplus Produce within 5km Geo-Fence
exports.getNearbySurplus = async (req, res) => {
  const { lat, lon } = req.query;
  const maxRadiusKm = process.env.MAX_SURPLUS_RADIUS_KM || 5;

  const surplusQuery = `
    SELECT id, crop_type, surplus_qty_kg, price_per_kg, latitude, longitude,
    (6371 * acos(cos(radians($1)) * cos(radians(latitude)) * cos(radians(longitude) - radians($2)) + sin(radians($1))
    * sin(radians(latitude)))) AS distance_km
    FROM surplus_listings
    WHERE status = 'AVAILABLE'
    HAVING (6371 * acos(cos(radians($1)) * cos(radians(latitude)) * cos(radians(longitude) - radians($2)) + sin(radians(
    $1)) * sin(radians(latitude)))) <= $3
    ORDER BY distance_km ASC;
  `;

  const result = await pool.query(surplusQuery, [lat, lon, maxRadiusKm]);
  res.json(result.rows);
};

```

Ravindi - Frontend Lead (React.js Dashboards)

- **Step 1:** Initialize React + Vite project with Tailwind CSS inside frontend/.
- **Step 2:** Configure protected routes based on decoded JWT roles.
- **Step 3:** Build Divisional Officer Dashboard displaying real-time saturation heatmaps.
- **Step 4:** Build Farmer Web Portal allowing pre-planting quota review.
- **Step 5:** Build Buyer Surplus Discovery View for rapid 5km distress procurement.
- **Step 6:** Set up Axios interceptors for automatic JWT Bearer token attachment.

```

import React, { useState, useEffect } from 'react';
import axios from 'axios';

export default function OfficerQuotaDashboard() {
  const [quotas, setQuotas] = useState([]);

  useEffect(() => {
    axios.get('http://localhost:5000/api/v1/officer/quotas?division_id=4')
      .then(res => setQuotas(res.data))
      .catch(err => console.error(err));
  }, []);

  return (
    <div className="p-6 max-w-5xl mx-auto">
      <h1 className="text-2xl font-bold text-gray-800 mb-2">Divisional Crop Saturation Monitor</h1>
      <p className="text-sm text-gray-600 mb-6">Uva Province - Badulla / Bandarawela District Division</p>

      <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
        {quotas.map(item => {
          const percent = Math.min(100, Math.round((item.planted_acres / item.quota_limit) * 100));
          const isWarning = percent >= 85;
          return (
            <div key={item.crop_type} className="p-4 bg-white border rounded shadow">
              <div className="flex justify-between items-center mb-2">
                <span className="font-bold text-lg">{item.crop_type}</span>
                <span className={`px-2 py-1 text-xs font-bold rounded ${isWarning ? 'bg-red-100 text-red-700' : 'bg-green-100 text-green-700'}`}>
                  {isWarning ? 'Saturation Alert' : 'Safe to Plant'}
                </span>
              </div>
              <div className="w-full bg-gray-200 h-4 rounded overflow-hidden mb-2">
                <div className={`h-full ${isWarning ? 'bg-red-500' : 'bg-blue-500'}`} style={{ width: `${percent}%` }}><
              </div>
              <p className="text-xs text-gray-500">{item.planted_acres} / {item.quota_limit} Acres Planned ({percent})%<
            </div>
          );
        })}
      </div>
    </div>
  );
}

```

Imal - Mobile Lead (Flutter Mobile App)

- **Step 1:** Scaffold Flutter app with BLoC, Dio, and Hive persistence.
- **Step 2:** Set API base URL to `http://10.0.2.2:5000/api/v1` for Android Emulator compatibility.
- **Step 3:** Build Pre-Planting Planning Screen to log crop intentions with instant alert toasts.
- **Step 4:** Build GPS Weather Advisory Screen consuming 30-min cached forecasts.
- **Step 5:** Build Emergency 5km Surplus Form to dispatch unexpected yields directly to local buyers.
- **Step 6:** Implement Hive local storage for seamless offline form logging in highland fields.

```
import 'package:dio/dio.dart';

class PlanningService {
  final Dio _dio = Dio(BaseOptions(
    baseUrl: 'http://10.0.2.2:5000/api/v1',
    connectTimeout: const Duration(seconds: 5),
  ));

  // 1. Submit pre-planting crop intention
  Future<Map<String, dynamic?>> submitCropPlan({
    required String cropType,
    required double plannedAcres,
    required String harvestDate,
  }) async {
    try {
      final response = await _dio.post('/plans', data: {
        'farmer_id': 'farmer_uva_001',
        'division_id': 4,
        'crop_type': cropType,
        'planned_area_acres': plannedAcres,
        'expected_harvest_date': harvestDate,
      });
      return response.data;
    } catch (e) {
      print('Error submitting crop plan: $e');
      return null;
    }
  }

  // 2. Offload emergency surplus to local 5km buyers
  Future<bool> postEmergencySurplus({
    required String cropType,
    required double surplusKg,
    required double pricePerKg,
    required double lat,
    required double lon,
  }) async {
    try {
      final response = await _dio.post('/surplus', data: {
        'crop_type': cropType,
        'surplus_qty_kg': surplusKg,
        'price_per_kg': pricePerKg,
        'latitude': lat,
        'longitude': lon,
      });
      return response.statusCode == 201;
    } catch (e) {
      print('Error posting surplus: $e');
      return false;
    }
  }
}
```

Miyuni - QA & Testing Lead

- **Step 1:** Create Postman workspace covering Planning, Quotas, and Surplus endpoints.
- **Step 2:** Implement automated assertions to verify that exceeding quota generates risk warnings.
- **Step 3:** Assert that surplus queries strictly filter out records beyond 5.0km.
- **Step 4:** Validate RBAC authorization returning HTTP 403 Forbidden for unauthorized quota edits.
- **Step 5:** Add backend Jest unit tests for Haversine distance math and acreage sums.
- **Step 6:** Execute full regression suite in Week 4 and produce the QA Sign-Off Report.

```

const request = require('supertest');
const app = require('../src/app');

describe('ASWANNA SRS Core Verification Suite', () => {
  it('1. Triggers saturation warning when divisional crop quota is exceeded', async () => {
    const res = await request(app).post('/api/v1/plans').send({
      farmer_id: 'farmer_001',
      division_id: 4,
      crop_type: 'Cabbage',
      planned_area_acres: 150,
      expected_harvest_date: '2026-11-15'
    });
    expect(res.statusCode).toEqual(201);
    expect(res.body.warning).not.toBeNull();
  });

  it('2. Filters surplus listings outside the 5km geo-fence radius', async () => {
    const res = await request(app).get('/api/v1/surplus/nearby?lat=6.95&lon=81.10');
    expect(res.statusCode).toEqual(200);
    res.body.forEach(listing => {
      expect(listing.distance_km).toBeLessThanOrEqual(5.0);
    });
  });

  it('3. Rejects non-officer role attempting to modify divisional quotas', async () => {
    const buyerToken = 'Bearer mock_buyer_token';
    const res = await request(app).put('/api/v1/officer/quotas/4').set('Authorization', buyerToken).send({
      quota_limit_acres: 500
    });
    expect(res.statusCode).toEqual(403);
  });
});

```

5. 4-Week Implementation Roadmap

Phase / Week	Core Deliverables & Milestones	Lead Owners
Week 1: Foundation	Docker environment up, PostgreSQL schema migrations, MongoDB TTL collections, Flutter project initialization.	Nirman (DevOps) Tharaka (Backend)
Week 2: Prevention Engine	Pre-planting planning API, regional quota saturation math, Officer Dashboard heatmap, Farmer mobile planning UI.	Tharaka (Backend) Ravindi (Web) Imal (Mobile)
Week 3: Recovery Engine	5km Haversine geo-fencing, emergency distress surplus forms, Buyer discovery view, OpenWeather 30-min cache proxy.	Tharaka (Backend) Ravindi (Web) Imal (Mobile)
Week 4: QA & Deployment	Full automated regression tests, Postman verification sign-off, Kubernetes cluster manifests, production sign-off.	Miyuni (QA) Nirman (DevOps)